



Variabili e tipi di dati

Massimo Esposito



Le variabili nei programmi



Sommario

1. Le variabili nei programmi
 2. Regole di utilizzo delle variabili
 3. Tipi di dati
- 

Il ruolo delle variabili

- Le **variabili** permettono di memorizzare e gestire dati nella memoria del computer durante l'esecuzione di un programma.
- Sono utilizzate per riferirsi a **valori**, come il prezzo di un articolo o la distanza tra due città, nella memoria del computer.
- Ogni variabile è un **nome simbolico** che punta a un'informazione specifica.

Istruzione di assegnazione

- Un'istruzione di assegnazione attribuisce un valore a una variabile usando il simbolo '=' (operatore di assegnazione).
- La sintassi generale è 'variabile = espressione', dove l'espressione può essere un numero, una somma o una funzione.
- Dopo l'esecuzione di un'istruzione di assegnazione, la variabile elencata sulla sinistra dell'operatore = fa riferimento al valore riportato alla destra dell'operatore =.

Esempio di variabile



- Se si scrive `età = 25`, il programma crea una variabile chiamata **età** che fa **riferimento** al valore **25**.
- La freccia che collega 'età' a 25 indica il **legame semantico** tra variabile e valore.
- Questo legame permette al programma di usare il valore ogni volta che si richiama la variabile.



Esempio di assegnazione multipla

`larghezza = 10`

`lunghezza = 5`

- La prima istruzione crea una variabile di nome **larghezza** e le assegna il valore 10.
- La seconda istruzione crea una variabile di nome **lunghezza** e le assegna il valore 5.

Esempio di assegnazione multipla

```
larghezza = 10  
lunghezza = 5
```

```
print(larghezza)  
print(lunghezza)
```

Output del programma:

```
10  
5
```

- Si può usare la funzione **print** per visualizzare i valori a cui fanno riferimento le variabili **larghezza** e **lunghezza**.
- Quando si passa una variabile come argomento alla funzione print, il nome della variabile **non va racchiuso fra virgolette**.

Esempio di assegnazione multipla

```
larghezza = 10
```

```
print('larghezza')  
print(larghezza)
```

Output del programma:

```
larghezza  
10
```

- Nella prima istruzione si è passato alla funzione print l'argomento **'larghezza'** e la funzione ha stampato la stringa larghezza.
- Nella seconda istruzione, si è passato alla funzione print l'argomento **larghezza** (senza virgolette) e la funzione ha stampato il valore a cui si riferisce la variabile larghezza.

Errore di sintassi nell'assegnazione

```
25 = età
```

```
# Output del programma:
```

```
SyntaxError: can't assign to literal
```

- In un'istruzione di assegnazione, la variabile che riceve l'assegnazione dev'essere sulla **sinistra dell'operatore =**.
- Se l'elemento alla sinistra dell'operatore = non è un nome di variabile si verifica un **errore di sintassi**.

Esempio di programma che usa variabili

```
# Il programma dimostra l'uso delle variabili.  
stanza = 503
```

```
print('Mi trovo nella stanza numero')  
print(stanza)
```

```
# Output del programma:  
Mi trovo nella stanza numero  
503
```

- Il programma crea una variabile chiamata **stanza** e le assegna il valore **503**.
- Le istruzioni successive visualizzano un messaggio, mostrando il **valore a cui fa riferimento la variabile stanza**.



Regole di utilizzo delle variabili

Uso di variabili non inizializzate

- Una variabile deve essere **inizializzata** con un valore prima del suo utilizzo.
- Se si tenta di eseguire un'operazione, come la stampa, su una variabile non ancora definita, il programma restituirà **un errore di esecuzione**.
- Questo tipo di errore è molto comune tra i principianti e può essere evitato verificando che ogni variabile sia stata correttamente assegnata prima del suo utilizzo.

Esempio di uso di variabili non inizializzate

```
temperature = 74.5 # Crea una variabile

print(tempereture) # Errore!
                  # Il nome della variabile è sbagliato

# Output del programma:
NameError: name 'tempereture' is not defined
```

- Non è possibile utilizzare la variabile **tempereture** perché non è mai stata inizializzata (**il nome è scritto in modo non coerente**).
- È facile incorrere in errori di battitura o confondere lettere maiuscole e minuscole nei nomi delle variabili.
- **In Python, i nomi delle variabili sono case-sensitive**, il che significa che anche **temperature** e **Temperature** sono variabili distinte.

Funzionamento interno delle variabili di Python

- In molti linguaggi, una variabile è una posizione in memoria che contiene **direttamente** un valore.
- In Python, invece, una variabile è una posizione in memoria che contiene **un riferimento** a un'altra posizione dove si trova effettivamente il valore.
- Si dice quindi che una variabile Python **“fa riferimento”** a un valore, e non che lo contiene direttamente.

Regole sintattiche e buone pratiche per i nomi delle variabili

Non è possibile usare **parole chiave riservate** del linguaggio come nomi di variabili.

I nomi non possono contenere **spazi** e devono iniziare con una **lettera** (a-z o A-Z) o **un trattino di sottolineatura** `_`.

- dopo il primo carattere, sono ammessi anche numeri e ulteriori sottolineature.

Python **fa distinzione tra maiuscole e minuscole**.

- i nomi di variabili `OggettiOrdinati` e `oggettiordinati` sono diversi.

Stili di denominazione per nomi delle variabili semplici

Oltre al rispetto delle regole sintattiche, è importante usare **nomi esplicativi** che chiariscano lo **scopo** della variabile.

- Invece di abbreviazioni generiche come x o b2, è preferibile usare nomi come temperatura o velocità_auto.

Stili di denominazione per nomi delle variabili complessi

- Per chiarire lo scopo, spesso si usano nomi composti di più parole per le variabili.
- Per migliorare la **leggibilità**, si possono separare le parole con trattini di sottolineatura (**retribuzione_lorda**).
- Un altro stile diffuso è il **camelCase**, dove la prima parola inizia con una lettera **minuscola** e le parole successive iniziano con una **maiuscola**, ad esempio **retribuzioneLorda**.

Esempi di nomi di variabile

Nome di variabile	Legale o illegale?
unita_al_giorno	Legale
giornoDellaSettimana	Legale
3dGraph	Illegale. I nomi di variabili non possono iniziare con un numero.
giugno1997	Legale
Misto#3	Illegale. I nomi di variabili possono contenere solo lettere, numeri o trattini di sottolineatura.

- In tabella sono riportati svariati esempi di nomi di variabile e viene indicato quali sono legali e quali illegali in Python.

Visualizzare più elementi con la funzione print

- In Python, è possibile semplificare la visualizzazione di più valori utilizzando **una sola chiamata alla funzione print**.
- Tradizionalmente si potrebbero usare più chiamate alla funzione print per stampare stringhe e variabili **distinte su righe separate**.
- Tuttavia, **passando più argomenti separati da virgole** a print, Python li stampa in sequenza, separandoli automaticamente con uno spazio.

Esempio di chiamata singola della funzione print

```
# Il programma dimostra l'uso delle variabili.  
stanza = 503
```

```
print('Mi trovo nella stanza numero', stanza)
```

```
# Output del programma:
```

```
Mi trovo nella stanza numero 503
```

- La chiamata `print("Mi trovo nella stanza numero", stanza)` è **preferibile rispetto a due chiamate separate**.
- Questa funzionalità rende il codice più **compatto e leggibile**, evitando la necessità di concatenare stringhe manualmente o gestire spazi a mano.

Il comportamento dinamico delle variabili

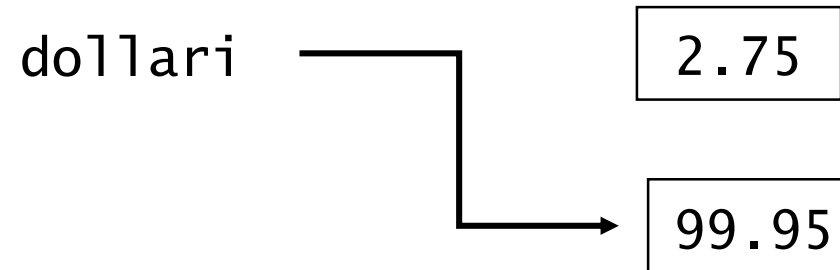
- Le variabili in Python prendono il nome dalla loro capacità di **far riferimento** a valori diversi durante l'esecuzione del programma.
- Quando si assegna un valore a una variabile, essa mantiene il riferimento a quel valore **fino a quando non le viene assegnato un nuovo valore**.

La garbage collection

La variabile dollari dopo l'esecuzione della riga 3.



La variabile dollari dopo l'esecuzione della riga 8.



- Se dollari = 2.75 e poi dollari = 99.95, la variabile smette di riferirsi al valore iniziale.
- Il valore 2.75 resta in memoria, ma poiché nessuna variabile vi fa più riferimento, viene rimosso **automaticamente** dall'interprete Python.
- Questo processo si chiama **garbage collection**, ed è fondamentale per l'efficienza e la gestione della memoria nei programmi Python.



Tipi di dati



Tipi di dati principali

- I linguaggi di programmazione distinguono diversi **tipi di dati**: numeri interi (**int**), numeri decimali (**float**), testi (**string**), valori **logici** (**boolean**).
- Ogni tipo ha **proprietà** e operazioni **compatibili**.
- L'uso corretto dei tipi di dati **garantisce precisione e robustezza** nei programmi.

Esempi di tipi di dati

- Un'età è rappresentata come **intero**: `età = 30`.
- Un prezzo può essere un **decimale**: `prezzo = 29.99`.
- Un nome è una **stringa**: `nome = 'Luca'`.
- Uno stato logico è un **booleano**: `acceso = True`.

Letterale numerico in Python

Un **letterale numerico** è un numero scritto direttamente nel codice, come 2.75 o 7.

In Python, ogni letterale numerico ha un tipo di dato assegnato **automaticamente** dall'interprete:

- Se è scritto senza punto decimale, come 7 o -9, è di tipo **int**.
- Se contiene un punto decimale, come 2.75 o 3.14159, è di tipo **float**.

Stringa in Python

```
# Crea variabili per fare riferimento a due stringhe.
```

```
nome = 'Maria'  
cognome = 'Bianchi'
```

```
# visualizza i valori a cui si riferiscono le variabili.  
print(nome, cognome)
```

```
# Output del programma:  
Maria Bianchi
```

- Oltre ai tipi numerici int e float, Python supporta anche il tipo di dato **str**, utilizzato per rappresentare le stringhe di testo.
- Le stringhe possono essere assegnate a variabili, **esattamente come i numeri**, permettendo al programma di manipolare testi, nomi, frasi o altri contenuti alfanumerici.

Conoscere il tipo di un valore

```
# Verifica il tipo dell'oggetto 1
```

```
type(1)
```

```
# Output del programma:
```

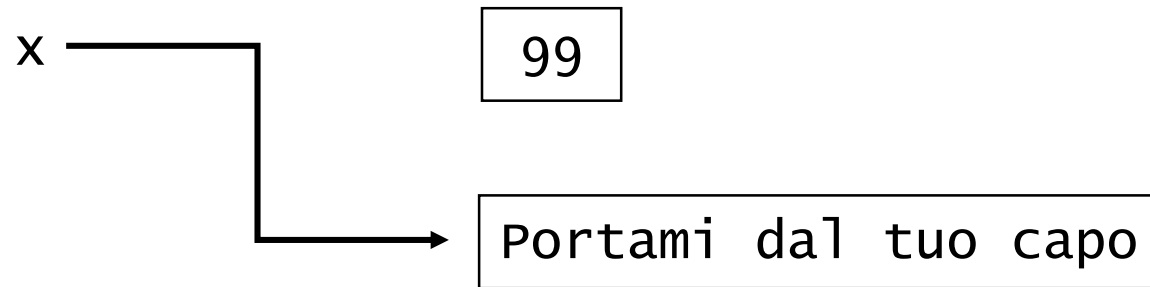
```
<class 'int'>
```

- Conoscere il tipo di dato è fondamentale, poiché alcune operazioni funzionano solo con **tipi specifici**.
- È possibile usare la funzione integrata **type()** per identificare il tipo di un valore e capire le operazioni ammissibili su esso.

Riassegnare una variabile a un tipo diverso

- In Python, una variabile è molto più di un semplice nome: è un meccanismo per memorizzare e recuperare dati in modo flessibile.
- L'interprete Python tiene traccia dei nomi delle variabili e dei valori a cui fanno riferimento, permettendo di accedere a questi valori in qualsiasi punto del programma.
- In Python, una variabile può essere **riassegnata** a dati di tipo **completamente diverso** rispetto a quelli iniziali.

Esempio di riassegnazione di una variabile



- Le variabili in Python sono **dinamiche**, cioè non legate a un tipo fisso come avviene in altri linguaggi.
- La variabile `x` può inizialmente riferirsi a un **intero** come `99` e successivamente a una **stringa** come `'Portami dal tuo capo.'`.

Bibliografia

Libro di testo

- Tony G. (2022). Introduzione a Python - 5/Ed (Capitolo 2, pp. 57-66). Pearson Italia spa. (Disponibile nella sezione “Biblioteca”)

Crediti

- Tutte le immagini utilizzate in questa presentazione sono tratte dal libro di testo qui indicato in bibliografia.